U.PORTO
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

SHOPIFY

Local-First Application

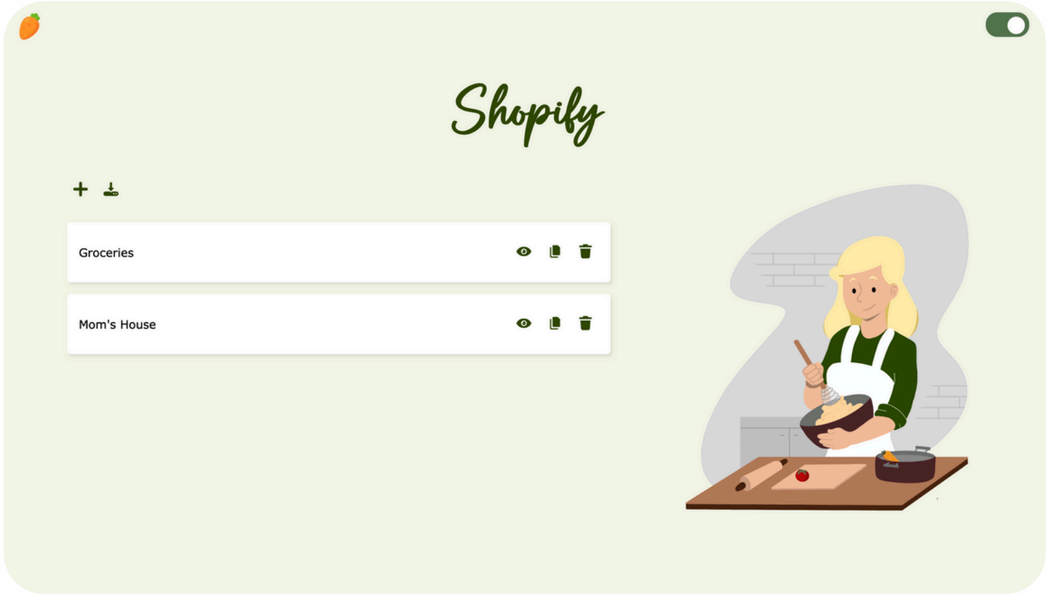
Pedro Angélico - up202108866@up.pt
Sofia Pinto - up202108682@up.pt
Tomás Gaspar - up202108828@up.pt

Goals

- Local - First Application
- Use CRDT’s for handling concurrency
- Avoid access data bottlenecks

Interface

Remote connection toggle



Home Page

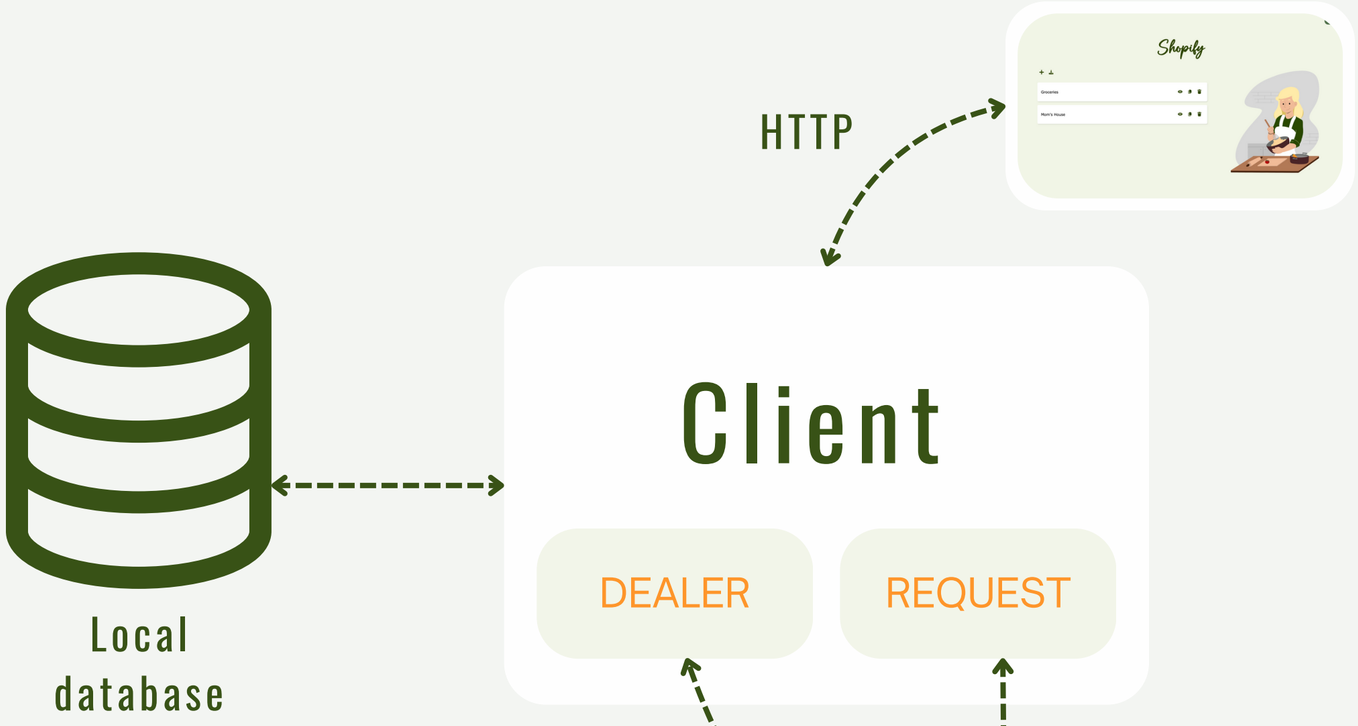
List View Page

Technologies

1. Node.js: TypeScript
2. Messaging: ZMQ
3. Database: SQLite
4. Packages

- uuid: Generate globally unique identifiers
- crypto: Calculate hashes for the hashring
- async-mutex: Ensure exclusive access to the database

Architecture - Client



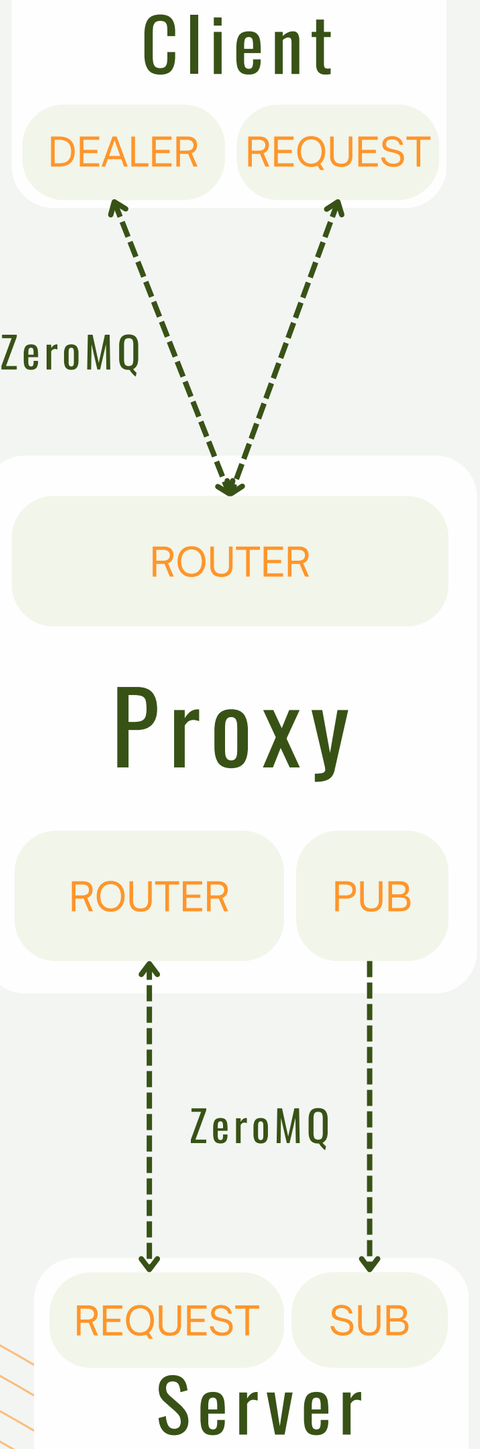
Client

- Makes use of CRDTs to achieve eventual consistency and seamless conflict resolution

- Dealer: Allows for asynchronous, non-blocking communication with the proxy. Used for sending updates and periodically asking for updates.

- Request: Synchronous communication for requesting new lists.

Architecture Proxy



servers.json

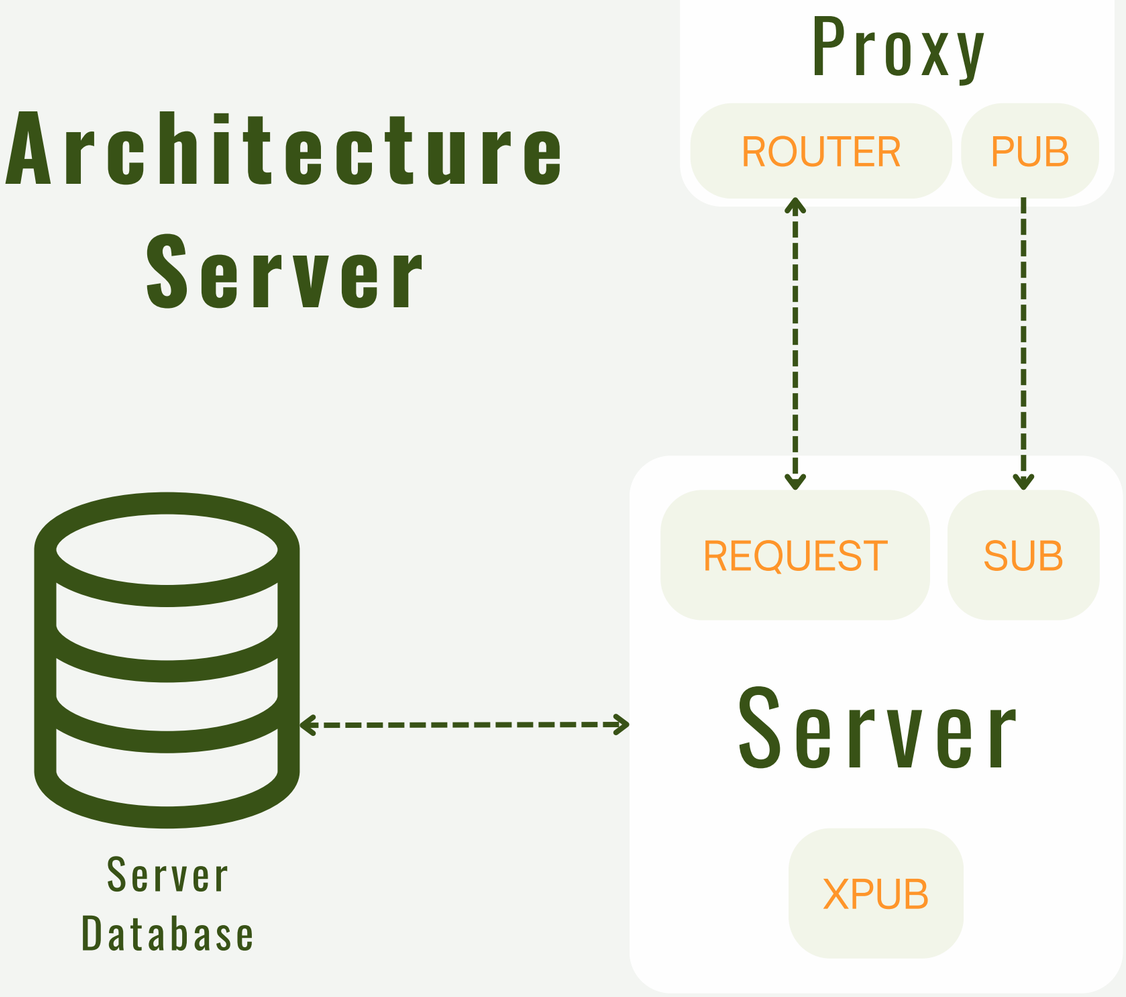
```
{
    "num_virtual_nodes": 3,
    "num_replicas": 3,
    "ports": [
        5000,
        5001,
        5002
    ]
}
```

Proxy

- Holds initial hashring configuration
- Acts as a load balancer
- Initiator of changes in the hashring
- Single Point of Failure, but simple and can easily recover from failures

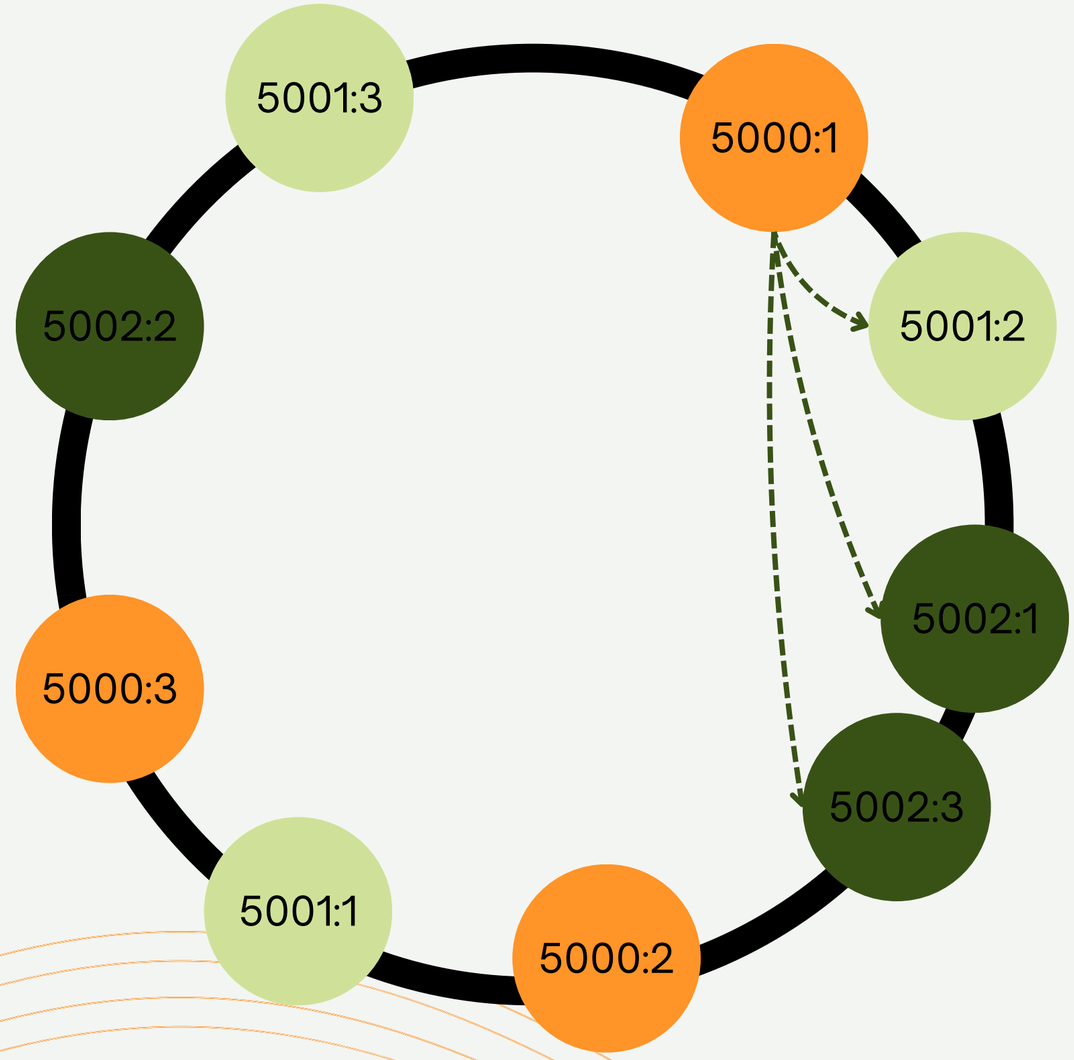
- Router: Forwards client requests to servers, and server replies to the client that requested them.
- Pub:
  - Notify servers that the proxy is (back) up
  - Publish changes in the hashring - up to the servers to make necessary changes

Architecture Server



- **Request:** Makes a request to the server indicating availability or returning the result of an operation. The reply from the proxy acts as a request to the server.
- **XPub:** Publishes updates to replicas. XPub allows to knwow when a new subscription happens so that we can retransmit the last few messages that were published.
- **Sub:** subscribes to replica and hashring updates

Hashring



- Each node subscribes to updates of the hashes they are responsible for
- When a node updates the contents of one of their replicas they publish the update so that others can synchronize
- Eventual consistency is achieved through the use of CRDTs
- When a hashring update is received, each node changes their internal view of the ring

CRDTs

Fundamental for ensuring data consistency and conflic resolution in local first applications

AWORStructure

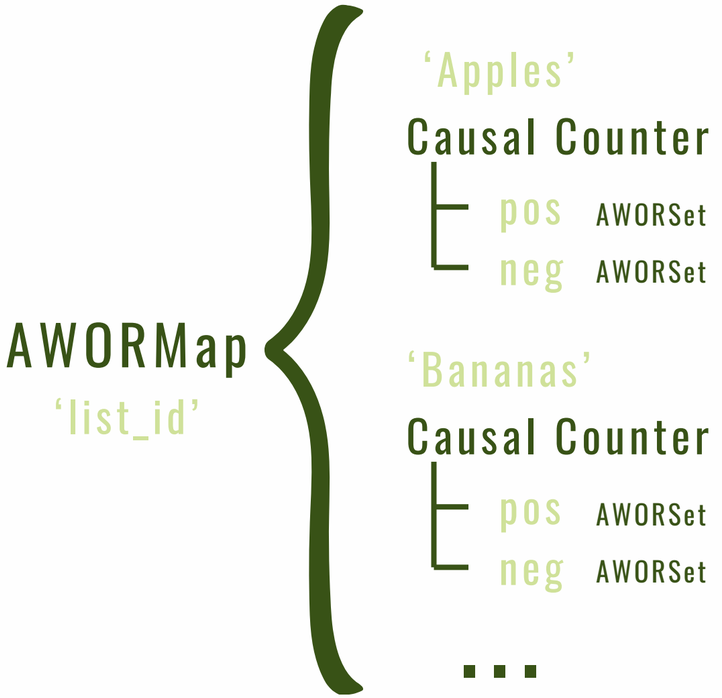
- AWORSet
- AWORMap

Add Wins
Observed Remove

- The addition of an element prevails over its removal

Causal Counter

- Uses AWORSets to keep track of increments and decrements



Conclusions

- [x] Many factors come into play when designing a distributed system, and there is often no perfect solution.
- [x] CRDTs are invaluable for achieving eventual consistency and developing local first applications.

THANK YOU!